

D7054E Lab 3

Abdalrhman Dabor

Group 16

March 9, 2026

1 Introduction

I used Python and the Scikit-Learn library to implement a machine learning model to address a classification problem. To evaluate the dataset and forecast the results, I decided to use the **Decision Tree algorithm**. "Decision Trees are easy to understand and interpret, mimicking the human level thinking. This allows us to see the logic for the decision made." [1]

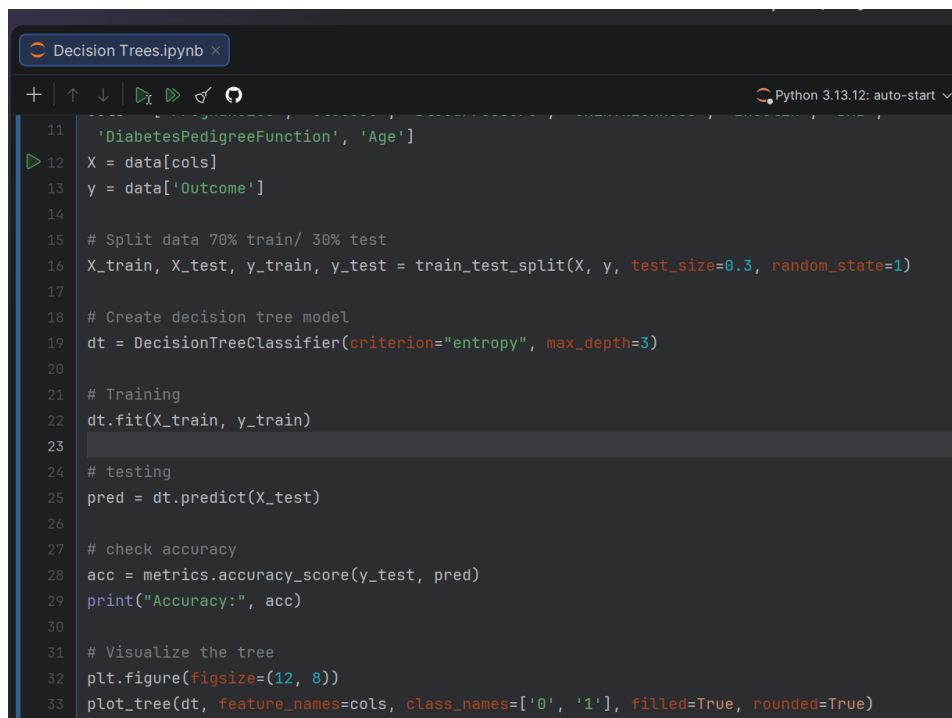
2 Algorithm Description

I selected the Decision Tree algorithm because it is easy to understand. It works by splitting the data step by step based on feature values until it reaches a final prediction.

- **Dataset:** diabetes.csv. "This dataset is originally from the National Institute of Diabetes and Digestive and Kidney Diseases. The objective is to predict whether or not a patient has diabetes based on diagnostic measurements." [2]
- **Goal:** Determine if a patient has diabetes using diagnostic metrics.

3 Implementation

In this lab, I used a virtual environment in Anaconda as required. Below are key snippets of the code:



```
11 'DiabetesPedigreeFunction', 'Age']
12 X = data[cols]
13 y = data['Outcome']
14
15 # Split data 70% train/ 30% test
16 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=1)
17
18 # Create decision tree model
19 dt = DecisionTreeClassifier(criterion="entropy", max_depth=3)
20
21 # Training
22 dt.fit(X_train, y_train)
23
24 # testing
25 pred = dt.predict(X_test)
26
27 # check accuracy
28 acc = metrics.accuracy_score(y_test, pred)
29 print("Accuracy:", acc)
30
31 # Visualize the tree
32 plt.figure(figsize=(12, 8))
33 plot_tree(dt, feature_names=cols, class_names=['0', '1'], filled=True, rounded=True)
```

Figure 1: Key snippets of the code

4 Results and Visualization

After splitting the data into 70% for training and 30% for testing, the model reached the following accuracy.

Accuracy: 0.7705627705627706

Figure 2: Decision tree accuracy

Visualization

The following plot shows the decision tree structure generated by the model:

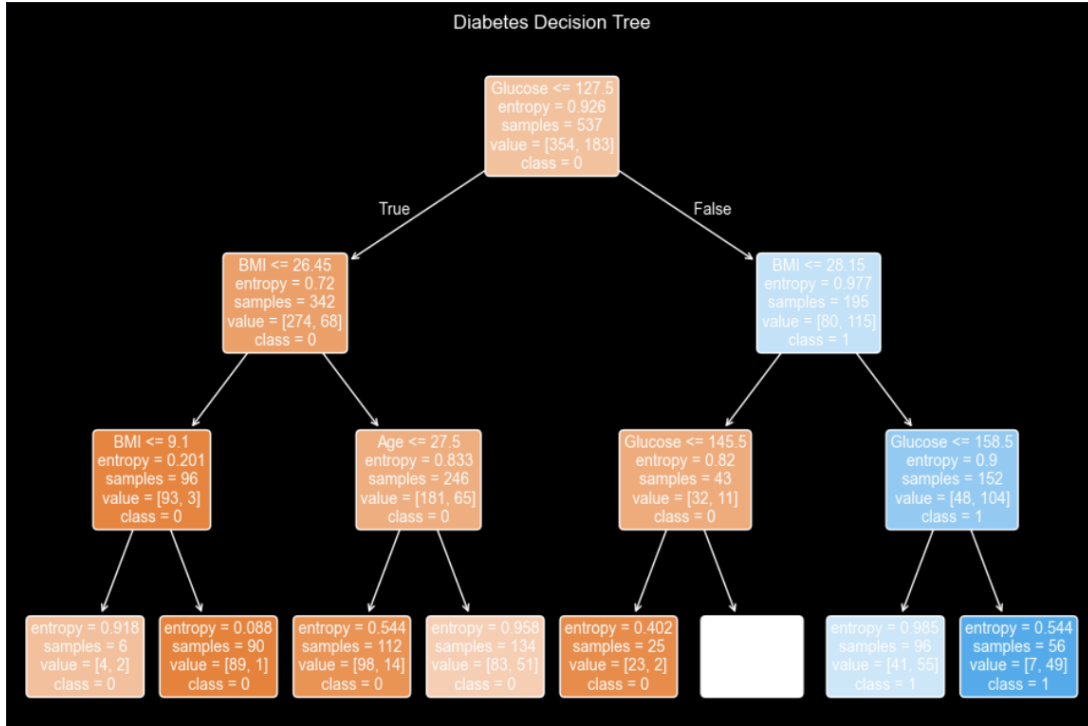


Figure 3: Decision tree structure

5 Discussion

The model selected Glucose as the root node, which suggests that it was the most important feature in the first split. Patients with lower glucose levels are typically categorized as non-diabetic, according to the visualization. This result shows that the model can help with this classification task, even though its accuracy is not perfect. From the visualized Decision Tree 3, we can see exactly how the model makes predictions for diabetes. Here are the main observations from the graph:

- The Root Node:** As can be seen from the graph, the most important factor in the detection of diabetes is the level of Glucose in the blood. The first split is based on whether the level of Glucose is less than or equal to 127.5.
 - Left Side (True):** If the level of Glucose is less than 127.5, the patient is likely to be in Class 0.
 - Right Side (False):** If the level of Glucose is greater than 127.5, the patient is more likely to have diabetes, and the decision tree moves to the right branch.
- Secondary Features (BMI and Age):** After checking Glucose, the model looks at BMI.
 - For the left branch of the decision tree, when the patient's level of Glucose is low and the patient's BMI is ≤ 26.45 , the model predicts "Class 0" with very high confidence.

- For the right branch of the decision tree, the model proceeds with the check for the patient's BMI and Age.
3. **The "White" Node** One leaf node appears less pure than the others. This means the samples in that node are more balanced between the two classes, so the model is less certain in that part of the tree.
 4. **Overall Performance:** The model used entropy to choose the splits, and limiting the depth to 3 helped keep the tree simple. The most important features in the tree were Glucose and BMI, which makes sense for diabetes prediction

References

- [1] Avinash Navlani, *Decision Tree Classification in Python Tutorial*. DataCamp, Updated Jan 16, 2026. Available at: <https://www.datacamp.com/tutorial/decision-tree-classification-python>
- [2] UCI Machine Learning Repository, *Pima Indians Diabetes Database*. Kaggle Dataset. Available at: <https://www.kaggle.com/datasets/uciml/pima-indians-diabetes-database>